Ingeniería de Software

Seminario de Arquitecturas de Software: Un Análisis Comparativo de Estilos Basados en Capas y Dominio

Equipo: Merxirve

Integrantes (C311):

- Enrique González González — C311
- Ernesto Javier Govea Varona — C311
- Olivia Ortiz Arboláez — C311
- Anthony Cruz Garcia — C311
- Jose Manuel Porras Guerra — C311

Pregunta	Responsable
1	Anthony Cruz Garcia
2	Olivia Ortiz Arboláez
3	Ernesto Javier Govea Varona
4	Enrique González González
5	Jose Manuel Porras Guerra
6	Enrique González González

Repositorio del proyecto:
https://github.com/Anthonycg2003/sistema-horarios-docentes

Índice

1. Análisis de trade-offs **3**
- 1.1. Enunciado . . . 3
- 1.2. Respuesta . . . 3
  - 1.2.1. ¿Qué error fundamental de acoplamiento se está cometiendo y cómo la introducción de la Regla de Dependencia (Martin) o el uso de Puertos y Adaptadores (Cockburn) resolverían estructuralmente este problema? . . . 3
  - 1.2.2. ¿Cuál sería el impacto en la testabilidad y la capacidad de cambiar la base de datos de SQL Server a MongoDB sin afectar la lógica de negocio? . . . 6
  - 1.2.3. Síntesis . . . 8

2. Hexagonal vs. Onion: simetría y jerarquía de capas **8**
- 2.1. Enunciado . . . 8
- 2.2. Respuesta . . . 8

3. DDD y contextos delimitados en un dominio de seguros **10**
- 3.1. Enunciado . . . 10
- 3.2. Respuesta . . . 10

4. Inversión de dependencias en la práctica **12**
- 4.1. Enunciado . . . 12
- 4.2. Respuesta . . . 13
  - 4.2.1. Violación de principios . . . 13
  - 4.2.2. Rediseño . . . 14
  - 4.2.3. Testabilidad y evolución . . . 15
  - 4.2.4. Síntesis . . . 16

5. Kitty Problem y principio abierto/cerrado **16**
- 5.1. Enunciado . . . 16
- 5.2. Respuesta . . . 16
  - 5.2.1. El problema: un requisito transversal que obliga a cambiar todos los servicios . . . 16
  - 5.2.2. ¿Por qué la división en servicios por verbos o entidades llevó a este problema? . . . 17
  - 5.2.3. ¿Cómo diseñarlo con DDD y Onion/Clean para que el cambio afecte a un solo componente? . . . 18
  - 5.2.4. Propuesta: `Rides` y `Kittens` como plugins . . . 19
  - 5.2.5. Cómo respeta la solución el OCP y qué papel juegan los adaptadores 20
  - 5.2.6. Síntesis . . . 21

6. N-Capas y atributos de calidad **22**
- 6.1. Enunciado . . . 22
- 6.2. Respuesta . . . 22
  - 6.2.1. Capas abiertas y *fast-lane reader*: beneficio local, coste estructural . 22
  - 6.2.2. El *sinkhole anti-pattern* y la regla 80/20 . . . 23
  - 6.2.3. Por qué Hexagonal y Clean son inherentemente más adecuadas . . . 24

6.2.4. Síntesis 25

Referencias bibliográficas **27**

Declaración de Uso de la Inteligencia Artificial **28**

1. Análisis de trade-offs (N-Capas vs. Clean/Onion/Hexagonal)

1.1. Enunciado

Un equipo de desarrollo está construyendo un sistema de gestión de inventario. Inicialmente, eligen una Arquitectura N-Capas tradicional por su simplicidad y familiaridad. Después de un año, el sistema se ha vuelto un monolito difícil de mantener, donde los cambios en la base de datos afectan la lógica de negocio y la interfaz de usuario, y las pruebas unitarias son lentas y frágiles.

Con base en los principios de la Arquitectura Limpia y la Arquitectura Hexagonal, explique detalladamente qué error fundamental de acoplamiento se está cometiendo y cómo la introducción de la Regla de Dependencia (Martin) o el uso de Puertos y Adaptadores (Cockburn) resolverían estructuralmente este problema. ¿Cuál sería el impacto en la testabilidad y la capacidad de cambiar la base de datos de SQL Server a MongoDB sin afectar la lógica de negocio?

1.2. Respuesta

1.2.1. ¿Qué error fundamental de acoplamiento se está cometiendo y cómo la introducción de la Regla de Dependencia (Martin) o el uso de Puertos y Adaptadores (Cockburn) resolverían estructuralmente este problema?

En una arquitectura N-Capas *clásica* (figura 1) hay una separación física en proyectos, módulos o ensamblados .

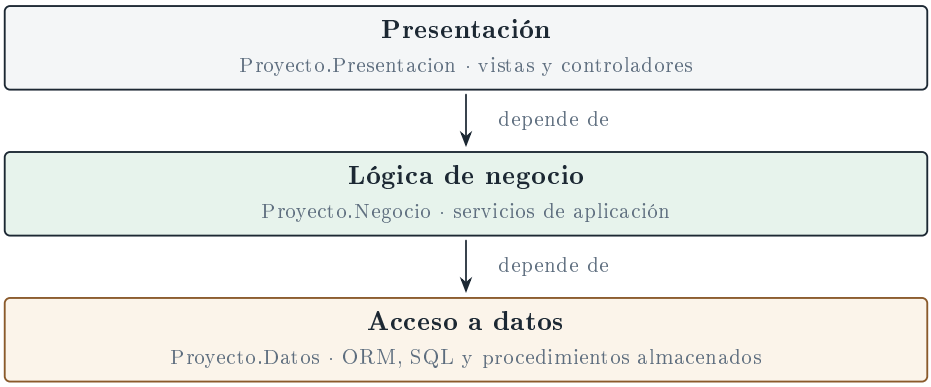


Figura 1: Arquitectura N-Capas clásica. Cada capa puede vivir en un ensamblado distinto, pero la dependencia de compilación apunta hacia el detalle inferior.

Esa separación no garantiza el aislamiento conceptual del dominio. Cuando las capas superiores dependen directamente de detalles concretos (ORM, consultas SQL, DTOs de persistencia, controles de interfaz), el **dominio queda acoplado a la infraestructura** y a los mecanismos de entrega (figura 2) (De la Torre Llorente et al., 2010; Martin, 2017).

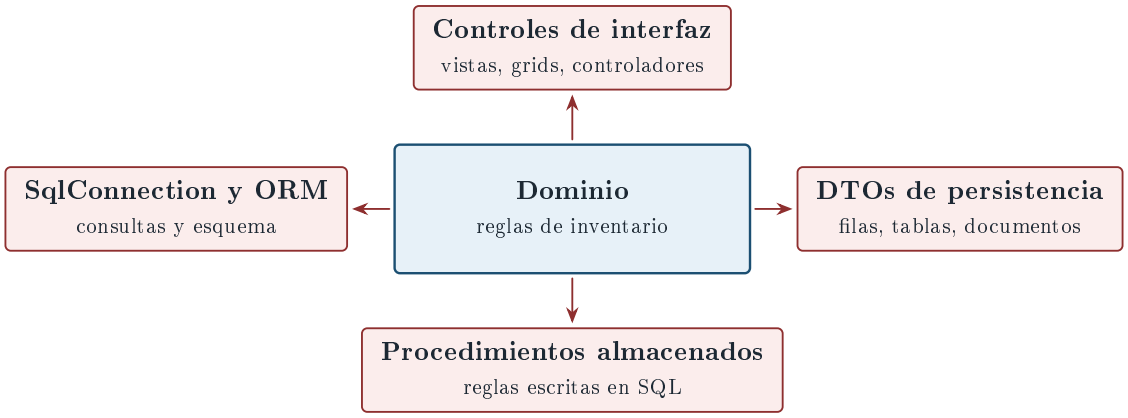


Figura 2: Error de acoplamiento. Las flechas salen del dominio: la lógica de inventario depende de la interfaz y de la persistencia, así que un cambio de esquema atraviesa el negocio y la UI. Elaboración propia.

El síntoma del enunciado (cambios en la base de datos que arrastran reglas de negocio y UI; pruebas lentas y frágiles) suele deberse a una violación combinada de:

- **Dependencias invertidas incorrectamente:** la lógica de negocio importa tipos del acceso a datos o del framework web, en lugar de que la infraestructura implemente contratos definidos por el dominio (Martin, 2017).
- **Modelo anémico y lógica dispersa:** las reglas de inventario viven en servicios de aplicación o en procedimientos almacenados, mientras las entidades son meros contenedores de datos, lo que difumina el límite entre "negocio" y "persistencia".
- **Acoplamiento transversal:** transacciones, validaciones y mapeos repetidos en cada capa generan un monolito lógico en el que ningún cambio local es verdaderamente local.

En términos de atributos de calidad, el sistema sacrifica *mantenibilidad* y *testabilidad* por una simplicidad inicial de despliegue en un solo proceso (Richards & Ford, 2020).

La Arquitectura Limpia organiza el software en anillos concéntricos: entidades y casos de uso en el centro; interfaces, persistencia y frameworks en el exterior (Martin, 2017). La **Regla de Dependencia** exige que todo vector de dependencia (importaciones, herencia, referencias) apunte *hacia adentro*. Nada en el núcleo puede conocer SQL Server, MongoDB, HTTP ni la UI (figura 3).

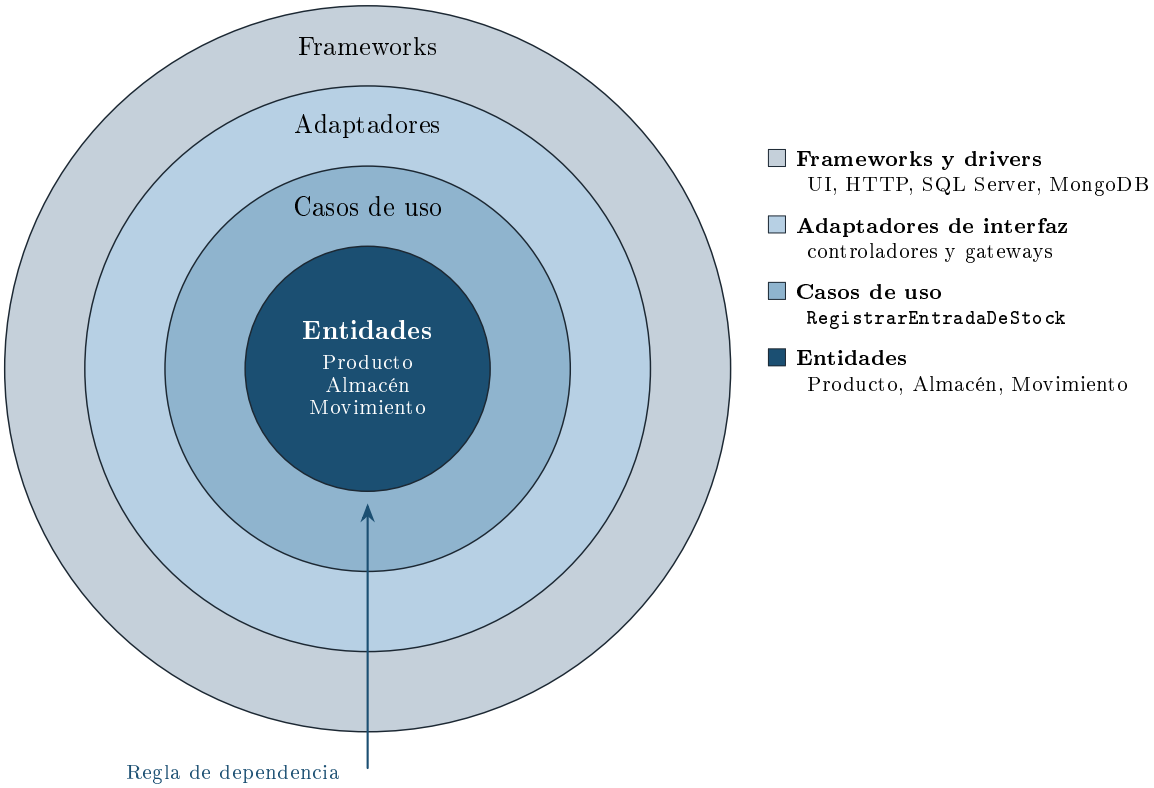


Figura 3: Regla de dependencia. El código solo puede depender hacia el centro: los casos de uso y las entidades no conocen SQL Server, MongoDB, HTTP ni la interfaz. Elaboración propia a partir de (Martin, 2017).

Aplicado al inventario, por ejemplo, el caso de uso *RegistrarEntradaDeStock* dependería de una abstracción (`IRepositorioDeProducto` o un puerto equivalente), no de `SqlConnection` ni de documentos BSON. La capa de datos implementa esa abstracción y traduce entre el modelo de dominio y el almacén concreto. Así, un cambio de esquema relacional a documentos no reescribe reglas de negocio: solo sustituye o añade un adaptador de salida (figura 4).

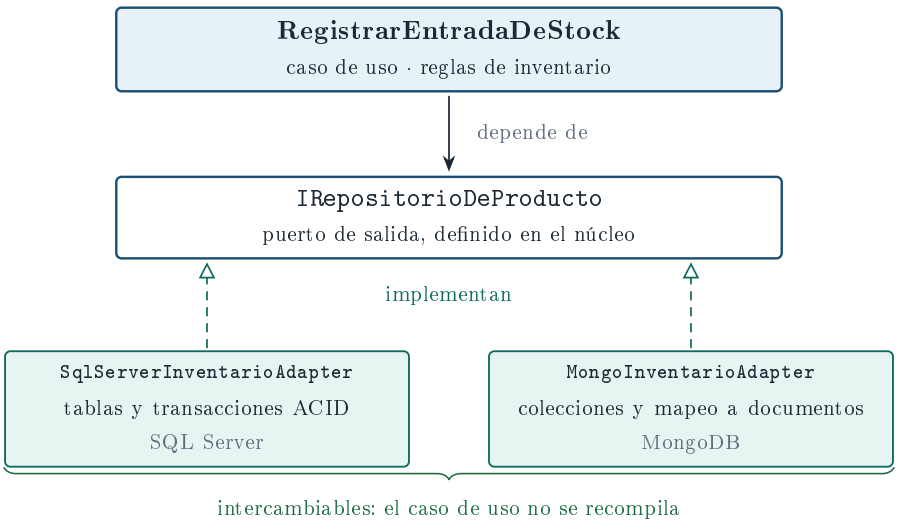


Figura 4: El caso de uso depende del puerto, no del motor. SQL Server y MongoDB quedan detrás de adaptadores que implementan la misma abstracción.

La Arquitectura Hexagonal expresa el mismo principio con la metáfora de **puertos** (interfaces que el dominio expone o requiere) y **adaptadores** (implementaciones concretas para UI, API, base de datos, mensajería) (Cockburn, 2005). El núcleo de la aplicación permanece aislado; los adaptadores son intercambiables.

- **Puerto de entrada:** por ejemplo, `GestionarInventario` invocado desde una API REST o una consola; ambos son adaptadores de entrada distintos sobre el mismo puerto.
- **Puerto de salida:** por ejemplo, `PersistirMovimientoDeInventario`; un adaptador habla con SQL Server y otro con MongoDB sin que el dominio distinga el motor.

Estructuralmente, esto rompe el acoplamiento en cadena "UI → servicio → repositorio concreto" y lo reemplaza por "adaptador → aplicación → puerto", con las dependencias cerradas hacia el centro (figura 5).

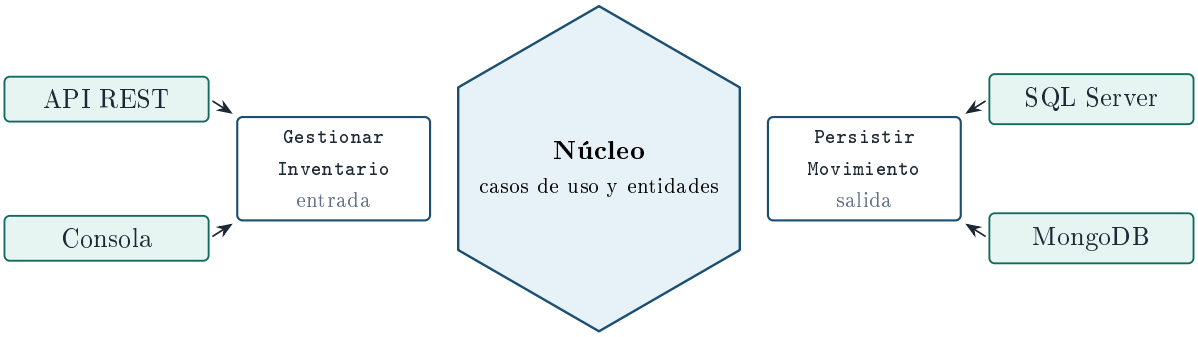


Figura 5: Puertos y adaptadores. La API y la consola usan el mismo puerto de entrada; SQL Server y MongoDB implementan el de salida. El núcleo no distingue el canal ni el motor. Elaboración propia a partir de (Cockburn, 2005).

1.2.2. ¿Cuál sería el impacto en la testabilidad y la capacidad de cambiar la base de datos de SQL Server a MongoDB sin afectar la lógica de negocio?

Con dependencias hacia adentro, los casos de uso y las entidades se prueban con **dobles de prueba** en memoria (repositorios falsos, relojes controlados, buses de eventos simulados) sin levantar base de datos ni servidor web (Cockburn, 2005; Martin, 2017). Las pruebas dejan de ser de integración pesada por defecto y pasan a ser unitarias rápidas y deterministas.

Además, al no filtrar detalles de persistencia al dominio, los tests dejan de romperse por cambios cosméticos en columnas o en el framework de UI: solo fallan cuando se altera un contrato de puerto o una regla de negocio explícita (figura 6).

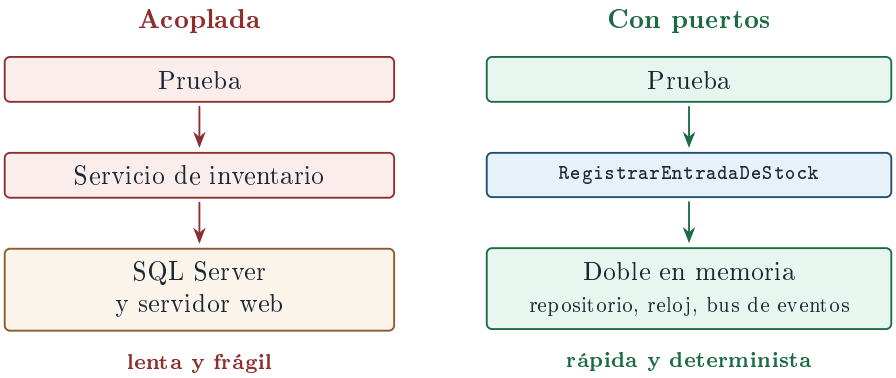


Figura 6: Efecto en la testabilidad. A la izquierda la prueba arrastra la base de datos y el servidor web. A la derecha el caso de uso se ejerce con dobles en memoria y no se entera de un cambio cosmético de columnas.

Supongamos que el dominio modela *Producto*, *Almacén* y *Movimiento* con invariantes (propiedades de stock no negativo, reservas coherentes, etc.). Bajo N-Capas acopladas, migrar a MongoDB implica retocar servicios, mapeos, validaciones y posiblemente pantallas que asumían identificadores o joins relacionales.

Con Regla de Dependencia y puertos/adaptadores:

1. Se define (o se mantiene) el puerto `IRepositorioDeInventario` en el núcleo.
2. El adaptador `SqlServerInventarioAdapter` implementa el puerto con tablas y transacciones ACID.
3. Un nuevo `MongoInventarioAdapter` implementa el mismo puerto con colecciones y, si hace falta, transacciones multi-documento; el mapeo documento–entidad vive solo en el adaptador.
4. La composición (inyección de dependencias en el arranque de la aplicación) selecciona el adaptador según configuración, sin recompilar el dominio.

La lógica de negocio permanece idéntica; el coste de migración se acota a infraestructura, pruebas de adaptador y estrategia de datos (dual write, backfill, corte), no a reescribir reglas de inventario (figura 7) (Martin, 2017; Richards, 2015).

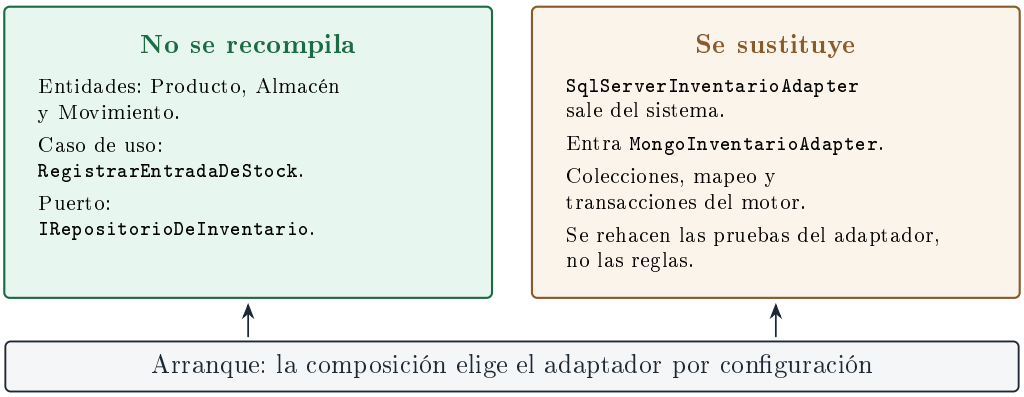


Figura 7: Límite del cambio de SQL Server a MongoDB. El núcleo y el puerto permanecen; el coste queda en el adaptador, en sus pruebas y en la estrategia de datos.

1.2.3. Síntesis

El error no es haber elegido N-Capas al inicio, sino permitir que las dependencias del dominio apunten hacia afuera hacia detalles volátiles. La Regla de Dependencia y los puertos/adaptadores no añaden capas por capricho: **invierten el flujo de control** para que el negocio mande sobre la tecnología. El resultado es un sistema más costoso de diseñar al principio, pero con pruebas rápidas y con la libertad de sustituir SQL Server por MongoDB (u otro almacén) sin erosionar el núcleo del inventario.

2. Hexagonal vs. Onion: simetría y jerarquía de capas

2.1. Enunciado

La Arquitectura Hexagonal de Cockburn y la Arquitectura Onion de Palermo comparten el objetivo de aislar el núcleo de negocio, pero utilizan representaciones visuales y metáforas radicalmente diferentes (un hexágono simétrico frente a capas concéntricas).

Analice en profundidad qué problema conceptual resuelve la simetría de la representación hexagonal que una representación de capas apiladas (como la de Onion o la N-Capas tradicional) no logra transmitir de forma intuitiva. ¿Cómo se traduce el concepto de «Puerto y Adaptador» en la práctica de definir interfaces en las capas internas de la Arquitectura Onion? ¿Por qué Cockburn afirma que el hexágono no tiene seis lados por una razón numérica, sino para eliminar la asimetría izquierda-derecha o arriba-abajo?

2.2. Respuesta

La metáfora visual elegida para representar una arquitectura condiciona profundamente la perspectiva de los desarrolladores respecto a las dependencias, el flujo de control y el desacoplamiento técnico (Cockburn, 2005; Martin, 2017; Palermo, 2008). A continuación, se analiza en detalle cada uno de los aspectos solicitados:

1. El problema conceptual que resuelve la simetría de la representación hexagonal

En las arquitecturas tradicionales N-Capas (Presentación arriba → Negocio al centro → Datos abajo), la disposición apilada introduce tres distorsiones conceptuales críticas:

1. **La ilusión de la Base de Datos como fundamento:** Colocar la capa de persistencia en la base del diagrama sugiere erróneamente que la base de datos es el cimiento o pilar sobre el cual descansa todo el software, condicionando el modelado de dominio a esquemas relacionales o tablas de almacenamiento (Cockburn, 2005; Martin, 2017).

2. **La prioridad jerárquica de la UI:** Ubicar la interfaz de usuario en la cima la presenta como el único punto de entrada «natural» del sistema (Cockburn, 2005).

3. **La simetría de la representación hexagonal:** Resuelve este dilema al establecer que, desde la perspectiva del núcleo de la aplicación (*Application Core*), **todos los componentes externos son simétricos e equivalentes** (Cockburn, 2005). No existe distinción funcional o jerárquica entre una interfaz gráfica web, una prueba

automatizada xUnit o una petición HTTP enviada por un tercero: todos son actores primarios (*drivers*) que invocan casos de uso. De la misma manera, una base de datos PostgreSQL, un broker de mensajes (como RabbitMQ) o una API externa son receptores/proveedores secundarios (*driven*) (Cockburn, 2005). El hexágono transmite de forma directa la frontera binaria fundamental: **Interior (lógica de negocio pura)** versus **Exterior (detalles tecnológicos y mecanismos de E/S)** (Cockburn, 2005; Martin, 2017).

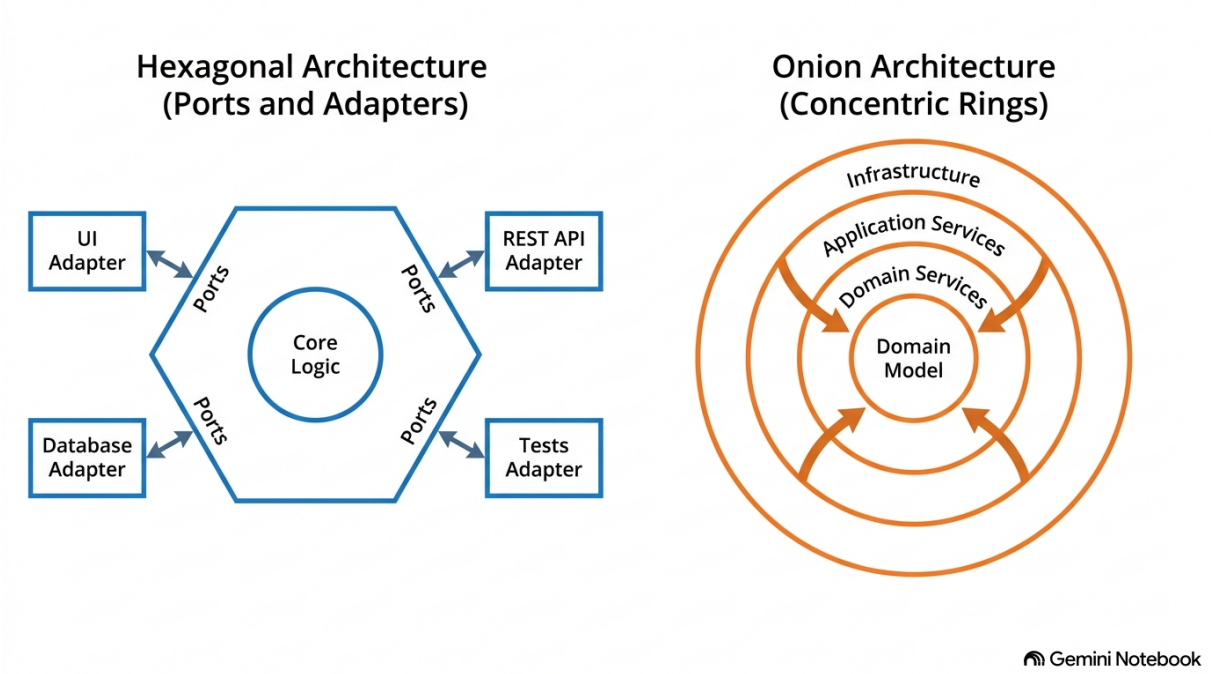


Figura 8: Comparación visual entre la simetría del hexágono y la jerarquía de capas de Onion.

2. Geometría del hexágono: eliminación de ejes de asimetría

Cockburn enfatiza que la elección de una figura geométrica de seis lados **no responde a una razón numérica** (el sistema no está limitado a seis componentes, seis puertos o seis tecnologías) (Cockburn, 2005). El propósito de utilizar un hexágono es **romper deliberadamente con la metáfora unidimensional** propia de las capas apiladas («arriba-abajo») o de los diagramas de tuberías («izquierda-derecha») (Cockburn, 2005). Un esquema rectangular tienta visualmente a encasillar «entradas por la izquierda» y «salidas por la derecha». El hexágono proporciona múltiples aristas simétricas e independientes para dibujar y distribuir puertos y adaptadores (UI, DB, servicios REST, tests, mensajes) en diversas direcciones alrededor del núcleo, evidenciando que el dominio no depende de ningún medio de transporte o almacenamiento en particular (Cockburn, 2005; Martin, 2017).

3. Traducción de «Puerto y Adaptador» en las capas internas de Onion

En la Arquitectura Onion de Palermo, el sistema se estructura en capas o anillos concéntricos alrededor del *Domain Model* y los *Domain Services*, los cuales son rodeados por los *Application Services* y la capa externa de *Infrastructure* (Palermo, 2008). El concepto

de **Puerto y Adaptador** se traduce operacionalmente en la Arquitectura Onion mediante la aplicación estricta del **Principio de Inversión de Dependencias (DIP)** y la **Regla de Dependencia** (Martin, 2017; Palermo, 2008):

- **Puerto ≡ Interfaz abstracta en capas internas (Core):** En Onion, un *Puerto* es un contrato de interfaz definido en las capas del centro (dentro del *Domain* o *Application Services*) (Cockburn, 2005; Palermo, 2008). Por ejemplo, la interfaz `IOrderRepository` o `IPaymentGateway` se declara dentro del núcleo y especifica *qué* requiere el dominio sin depender de detalles de implementación (Palermo, 2008).
- **Adaptador ≡ Clase concreta en la capa externa (Infrastructure):** Un *Adaptador* es la implementación concreta de dicha interfaz, la cual reside exclusivamente en el anillo periférico de *Infrastructure* o *UI* (Cockburn, 2005; Palermo, 2008). Por ejemplo, `SqlOrderRepository` (que implementa `IOrderRepository`) o `StripeAdapter`.

Dado que en Onion todas las dependencias del código fuente apuntan obligatoriamente hacia el centro (Palermo, 2008), la infraestructura externa depende de la interfaz definida en el núcleo interno. Así, se garantiza la total independencia y testabilidad del dominio frente a cambios tecnológicos (Martin, 2017; Palermo, 2008).

3. DDD y contextos delimitados en un dominio de seguros

3.1. Enunciado

Una empresa de seguros necesita desarrollar un sistema para gestionar pólizas de automóviles. El sistema debe interactuar con un departamento de siniestros, un sistema de facturación de primas y una aplicación para que los ajustadores de seguros evalúen daños en el lugar del accidente. Cada uno de estos subsistemas tiene su propia terminología y preocupaciones (por ejemplo, «póliza» para un actuario es un contrato con cláusulas, para facturación es una fuente de ingresos, y para un ajustador es un número de referencia).

Basado en los conceptos de Domain-Driven Design (Evans) y Bounded Context (Richards & Ford), explique cómo dividiría este sistema en varios contextos delimitados. ¿Cómo evitaría que el modelo de «Póliza» se contamine con detalles de siniestros o facturación? ¿Cómo se comunicarían estos contextos de manera desacoplada? Relacione esta solución con la implementación de una Arquitectura Orientada al Dominio (como Onion o Clean) y la creación de un Adaptador de Servicio para la comunicación externa, tal como se describe en la Guía de Arquitectura N-Capas.

3.2. Respuesta

División en contextos delimitados. El enunciado ya muestra la señal que Evans usa para trazar fronteras: la misma palabra («póliza») significa cosas distintas según quién la use. En DDD eso indica que hay varios *lenguajes ubicuos*, y cada uno debe vivir en su propio *bounded context*, con su propio modelo, su propio código y, según Richards y Ford, también sus propios datos, de modo que ningún otro contexto acceda directamente a ellos (Richards & Ford, 2020). Propondría cuatro contextos:

- **Suscripción / Pólizas** (*core domain*): la póliza como contrato. Entidad raíz `Poliza` con coberturas, cláusulas, vehículo asegurado, vigencia y reglas actuariales. Es el contexto propietario del concepto.
- **Siniestros**: el `Siniestro` (reclamación) con su estado, documentación y resolución. De la póliza solo necesita saber si estaba vigente y qué coberturas aplican en la fecha del accidente.
- **Facturación de primas**: la póliza vista como `CuentaDeCobro` o `ContratoFacturable`: importe de la prima, plan de pagos, recibos y morosidad.
- **Peritaje en campo** (aplicación de ajustadores): la `Inspeccion` con fotos, daños evaluados y estimación de reparación. La póliza aparece solo como un número de referencia (`PolizaRef`).

Las relaciones entre ellos se documentan en un *mapa de contextos*: Pólizas actúa como *upstream* (proveedor) de Siniestros y Facturación, y Peritaje es *downstream* de Siniestros.

Evitar la contaminación del modelo de Póliza.

1. **Un modelo por contexto, sin modelo canónico compartido.** No se crea una clase `Poliza` «universal» con campos de facturación y siniestros. Cada contexto define su propia representación mínima. En Siniestros, por ejemplo, existe un objeto de valor `CoberturaVigente(numeroPoliza, coberturas, desde, hasta)`, no la entidad completa.
2. **Referencias por identidad.** Los agregados de otros contextos guardan solo el identificador (`numeroPoliza`), nunca una referencia de objeto ni una clave foránea hacia tablas ajenas. Así se respeta la regla de Richards y Ford de que cada contexto es dueño exclusivo de sus datos (Richards & Ford, 2020).
3. **Capa anticorrupción (ACL).** Cuando un contexto consume datos o eventos de otro, los traduce en la frontera a su propio lenguaje. Si cambia el modelo de Pólizas, solo cambia el traductor y no el dominio consumidor.
4. **Lenguaje publicado.** Pólizas expone un contrato estable (DTO o esquema de evento versionado) que es distinto de su modelo interno. El modelo interno puede evolucionar sin romper a los consumidores.

Comunicación desacoplada. La vía principal es asíncrona, mediante **eventos de dominio** publicados en un bus o broker de mensajes:

- Pólizas publica `PolizaEmitida`, `PolizaRenovada` y `PolizaCancelada`. Facturación reacciona generando el plan de cobro, y Siniestros mantiene una réplica local de solo lectura de las coberturas vigentes (consistencia eventual).
- Siniestros publica `SiniestroRegistrado`. Peritaje crea a partir de él una orden de inspección y, al terminar, publica `DañosEvaluados`, que Siniestros consume para resolver la reclamación.
- Facturación publica `PrimaImpagada`. Pólizas puede decidir suspender la cobertura sin que Facturación conozca las reglas del contrato.

Cuando hace falta una respuesta inmediata (por ejemplo, el ajustador consulta en campo si la póliza cubre el daño), se usa una llamada síncrona a un servicio expuesto por el contexto propietario a través de su lenguaje publicado y protegida por la ACL del consumidor. Ningún contexto llama directamente a las clases ni a la base de datos de otro.

Relación con la arquitectura orientada al dominio y los adaptadores de servicio. Dentro de cada contexto delimitado se aplica una arquitectura en capas orientada al dominio (Onion, Clean o hexagonal), como propone la Guía de Arquitectura N-Capas (De la Torre Llorente et al., 2010):

- **Núcleo de dominio**: entidades, agregados, objetos de valor, servicios de dominio y eventos (`Poliza`, `Cobertura`, `PolizaEmitida`). No depende de ningún framework, base de datos ni mensajería.
- **Capa de aplicación**: casos de uso (`EmitirPoliza`, `RegistrarSiniestro`) que coordinan el dominio y definen *puertos* (interfaces) como `IRepositorioPolizas`, `IPublicadorEventos` o `IServicioCoberturas`.
- **Infraestructura**: implementa esos puertos (repositorios ORM, cliente del broker). Todas las dependencias apuntan hacia el centro (inversión de dependencias), y por eso un detalle técnico o de otro contexto no puede filtrarse al dominio.
- **Capa de servicios distribuidos**: expone el contexto al exterior mediante servicios web o API y DTO, sin publicar las entidades del dominio.

El **Adaptador de Servicio** de la guía N-Capas es la pieza de infraestructura que materializa la ACL. Implementa un puerto definido por el dominio consumidor, invoca al servicio o consume el evento externo, y traduce los DTO externos al modelo propio. Por ejemplo, en Siniestros:

- el dominio declara `IServicioCoberturas.obtenerCobertura(numeroPoliza, fecha)` → `CoberturaVigente`;
- `AdaptadorServicioPolizas` (infraestructura) llama al servicio de Pólizas, recibe su DTO y lo convierte en `CoberturaVigente`;
- si Pólizas cambia de protocolo o de contrato, o se sustituye por un sistema externo heredado, solo se modifica el adaptador.

Así, la estrategia de DDD (contextos delimitados y mapa de contextos) fija las fronteras entre subsistemas, y la arquitectura en capas orientada al dominio, junto con los adaptadores de servicio, protege cada frontera en el código. Cada contexto conserva un modelo coherente con su lenguaje, puede evolucionar y desplegarse de forma independiente y se comunica con los demás solo mediante contratos explícitos.

4. Inversión de dependencias en la práctica

4.1. Enunciado

Un equipo de desarrollo está trabajando en un sistema de gestión de pedidos. La lógica de negocio principal se encuentra en una clase `OrderProcessor` que, para completar

su tarea, necesita comunicarse con un servicio externo de validación de direcciones. La implementación actual tiene una dependencia directa: `OrderProcessor` instancia y llama directamente a la clase `AddressValidatorService`, que a su vez depende de una librería HTTP específica para comunicarse con el servicio externo.

Con base en los principios de la Inversión de Dependencias (Martin) y la Regla de Dependencia (Martin), analice y responda:

- **a)** ¿Por qué esta dependencia directa viola los principios fundamentales de las arquitecturas Hexagonal, Limpia y Onion? Explique específicamente qué dirección deberían tener las dependencias del código fuente y por qué la implementación actual no la cumple.
- **b)** Rediseñe la interacción entre `OrderProcessor` y `AddressValidatorService` para que se alinee con los principios de estas arquitecturas. Defina claramente qué componente (interfaz) debe residir en el núcleo de la aplicación y qué componente debe residir en la capa de infraestructura o en el exterior del hexágono. Justifique su diseño basándose en el principio de que «los detalles deben depender de las abstracciones».
- **c)** Explique cómo este nuevo diseño permite que la lógica de negocio (`OrderProcessor`) sea probada de forma aislada, sin necesidad de realizar llamadas HTTP reales al servicio externo, y sin depender de la librería HTTP subyacente. ¿Qué implicaciones tiene este diseño para la mantenibilidad y la capacidad de cambiar el proveedor de validación de direcciones en el futuro?

4.2. Respuesta

4.2.1. Violación de principios

La implementación actual viola el **Principio de Inversión de Dependencias** (DIP) y, con él, la **Regla de Dependencia** que articula las arquitecturas Hexagonal, Limpia y Onion. `OrderProcessor`, que contiene la lógica de negocio de alto nivel, instancia y llama directamente a `AddressValidatorService`, una clase de infraestructura que a su vez depende de una biblioteca HTTP concreta.

El DIP establece que los módulos de alto nivel no deben depender de los módulos de bajo nivel; ambos deben depender de abstracciones, y las abstracciones no deben depender de los detalles. Martin formula la Regla de Dependencia de Clean Architecture en esa misma dirección: ningún módulo de código debería importar a otro más cercano a los detalles, porque las flechas de dependencia apuntan siempre hacia las políticas y nunca hacia la infraestructura (Martin, 2017).

En la implementación actual, las dependencias apuntan **hacia afuera**: el núcleo de negocio conoce y depende de un detalle de infraestructura (HTTP, un proveedor concreto). En términos de la Arquitectura Hexagonal, el hexágono no debería conocer sus adaptadores externos. Cockburn (2005) concibe este estilo para que el núcleo de una aplicación ignore quién o qué la invoca —una persona, otro programa, una prueba automatizada o un script— y para que pueda desarrollarse y probarse al margen del dispositivo o la base de datos que usará en producción (Cockburn, 2005).

La Arquitectura Onion comparte la misma regla estructural: el acoplamiento debe dirigirse siempre hacia el interior del modelo, nunca hacia el exterior. Lo que está más cerca del borde puede depender del centro, pero no al revés (Palermo, 2008).

La dependencia directa actual impide ese aislamiento: acopla la lógica de negocio a un detalle tecnológico concreto, de modo que cualquier cambio en el mecanismo de validación (o en la biblioteca HTTP subyacente) impacta directamente en el núcleo. La figura 9 contrasta ambos escenarios.

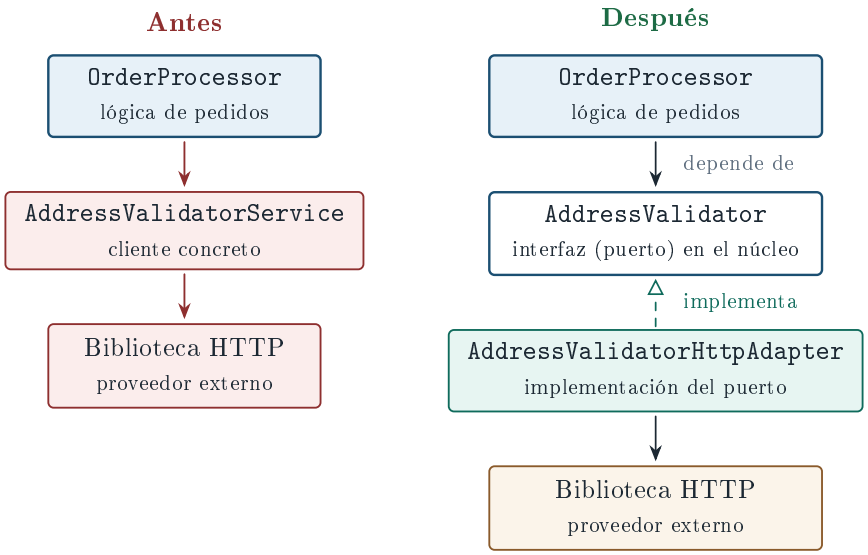


Figura 9: Antes y después de invertir la dependencia. A la izquierda, el núcleo conoce el detalle HTTP. A la derecha, el núcleo define el puerto y el adaptador de infraestructura lo implementa. Elaboración propia.

4.2.2. Rediseño

El rediseño se articula en dos componentes claramente separados:

1. **Componente en el núcleo (puerto):** se define una interfaz `AddressValidator` en la capa de aplicación/dominio. Esta interfaz declara el método necesario (por ejemplo, `validate(address): ValidationResult`) y expresa *qué* necesita la lógica de negocio, sin especificar *cómo* se obtiene. `OrderProcessor` depende únicamente de esta abstracción.

2. **Componente en infraestructura (adaptador):** se crea una clase denominada `AddressValidatorHttpAdapter` que implementa `AddressValidator` utilizando la biblioteca HTTP concreta para comunicarse con el servicio externo. Este adaptador reside fuera del hexágono, en la capa de infraestructura.

La justificación de este diseño es el principio de que **los detalles deben depender de las abstracciones**, no al revés. La abstracción (`AddressValidator`) reside en el núcleo y no conoce HTTP ni ningún proveedor concreto; mientras que la implementación (`AddressValidatorHttpAdapter`) depende de esa abstracción. Esto alinea el diseño con la regla de la Arquitectura Onion: la base de datos y los servicios externos quedan fuera del núcleo, y el acoplamiento apunta siempre hacia el centro (Palermo, 2008).

El mecanismo concreto para que `OrderProcessor` reciba la implementación sin instanciarla es la **inyección de dependencias**. Existen tres variantes habituales (figura 10):

- **Por constructor:** `OrderProcessor(AddressValidator validator)`. Es la preferida en arquitecturas limpias porque hace explícitas las dependencias y permite que el objeto sea inmutable respecto a ellas.

- **Por método (o setter):** `orderProcessor.setValidator(validator)`. Útil cuando la dependencia es opcional o cambia en tiempo de ejecución, a costa de perder la garantía de que el objeto siempre está completamente inicializado.
- **Por interfaz o fábrica:** el núcleo declara una fábrica (`AddressValidatorFactory`) y la infraestructura la implementa. Se usa cuando la creación del adaptador requiere configuración que el núcleo no debe conocer.

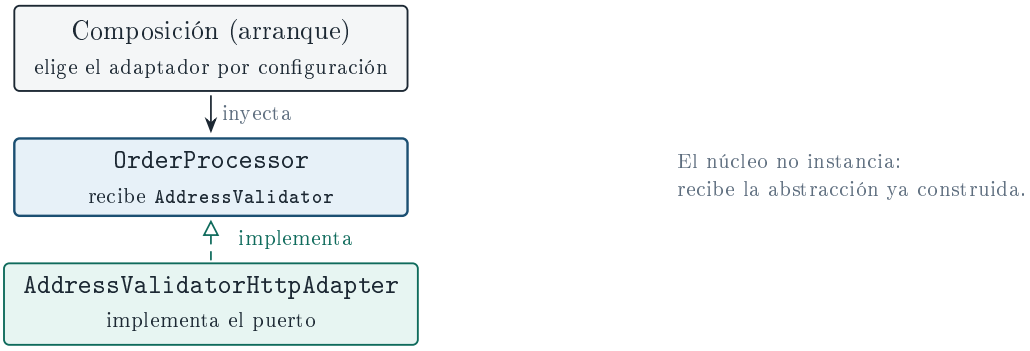


Figura 10: Inyección por constructor. La composición (infraestructura) construye el adaptador y lo pasa al núcleo. `OrderProcessor` nunca escribe `new AddressValidatorHttpAdapter()`. Elaboración propia.

4.2.3. Testabilidad y evolución

Con el nuevo diseño, `OrderProcessor` puede probarse de forma aislada inyectando una implementación falsa (*fake*) o un *mock* de `AddressValidator`. Las pruebas unitarias del núcleo no realizan llamadas HTTP reales, no dependen de la red ni de la biblioteca HTTP subyacente: son rápidas, deterministas y verifican exclusivamente la lógica de negocio. Esto es posible porque, como señala Cockburn (2005), la arquitectura hexagonal permite que la aplicación sea desarrollada y probada en aislamiento de sus eventuales dispositivos y bases de datos en tiempo de ejecución.

En cuanto a la mantenibilidad, cambiar el proveedor de validación de direcciones (por ejemplo, de un servicio HTTP a una base de datos local o a otro proveedor externo) solo requiere crear un nuevo adaptador que implemente `AddressValidator`. El núcleo —`OrderProcessor` y la interfaz `AddressValidator`— no se modifica. Esto materializa el objetivo del DIP: las políticas de alto nivel no dependen de los detalles; son los detalles los que dependen de las abstracciones (Martin, 2017). Como consecuencia, el sistema gana en mantenibilidad y capacidad de evolución, ya que los cambios tecnológicos quedan confinados a la capa de infraestructura, siguiendo la regla de dependencia hacia el centro propia de las arquitecturas orientadas al dominio (Palermo, 2008).

Este efecto sobre los atributos de calidad es consistente con lo documentado en la guía de arquitectura N-Capas orientada al dominio, donde la separación estricta entre dominio e infraestructura se presenta como condición para que la lógica de negocio sea independiente de la tecnología de persistencia y de los mecanismos de entrega (De la Torre Llorente et al., 2010). Asimismo, desde la perspectiva de los atributos de calidad, la testabilidad y la mantenibilidad son precisamente los atributos que se deterioran cuando el acoplamiento arrastra la infraestructura al núcleo (Richards & Ford, 2020).

4.2.4. Síntesis

El error no es haber elegido una dependencia directa por simplicidad, sino permitir que las dependencias del núcleo apunten hacia detalles volátiles. La Regla de Dependencia y los puertos/adaptadores no añaden capas por capricho: **invierten el flujo de control** para que la lógica de negocio mande sobre la tecnología. El resultado es un sistema algo más costoso de diseñar al principio, pero con pruebas unitarias rápidas y con la libertad de sustituir el proveedor de validación (o la biblioteca HTTP subyacente) sin erosionar el núcleo.

5. Kitty Problem y principio abierto/cerrado

5.1. Enunciado

En su libro *Fundamentals of Software Architecture*, Richards y Ford presentan el «Kitty Problem» para ilustrar un fallo común en el diseño de arquitecturas orientadas a servicios. Describa este problema: un requisito transversal (como la entrega de gatitos) afecta a múltiples servicios previamente existentes (TaxiUI, TaxiFinder, TaxiSelector, etc.), requiriendo cambios coordinados en todos ellos.

¿Por qué la solución de dividir el sistema en servicios basados en verbos o entidades funcionales (como TaxiFinder o TaxiSelector) llevó a este problema? Basándose en los principios de DDD y la Arquitectura Onion/Clean, ¿cómo se podría haber diseñado el sistema para que un cambio de este tipo afectara a un solo componente o servicio? Proponga una solución utilizando componentes como Kittens y Rides que se añadan como plugins, explicando cómo esto respeta el Principio de Abierto/Cerrado (OCP) y qué papel juegan los Adaptadores en esta extensión.

5.2. Respuesta

El *Kitty Problem* muestra que repartir un sistema en servicios no basta para desacoplarlo. Si los servicios siguen los pasos de un flujo (buscar, elegir, encargar), cualquier requisito que cambie la naturaleza del producto atraviesa todos los pasos y obliga a cambiarlos a la vez. La salida es trazar las fronteras según los ejes de cambio del negocio, con un núcleo estable que defina puntos de extensión y con cada producto (`Rides`, `Kittens`) como un componente que se enchufa a ese núcleo.

Aclaración sobre la fuente. El problema no aparece en el libro de Richards y Ford: lo presenta Martin en el capítulo 27 de *Clean Architecture*, «Services: Great and Small» (Martin, 2017, pp. 242–244). Richards y Ford (2020, caps. 8 y 17) sí tratan ideas afines que se usan más abajo, como la trampa de las entidades y el contexto delimitado como base de la granularidad de los servicios.

5.2.1. El problema: un requisito transversal que obliga a cambiar todos los servicios

Martin parte de un agregador de taxis que conoce a muchos proveedores de una ciudad y permite a los clientes pedir viajes. Para que fuera escalable, se construyó con muchos microservicios pequeños, repartidos entre equipos pequeños, cada uno responsable de unos pocos servicios. `TaxiUI` atiende a los clientes; `TaxiFinder` revisa los inventarios

de los `TaxiSuppliers` y deja los candidatos en un registro temporal asociado al usuario; `TaxiSelector` elige entre ellos según los criterios del cliente (coste, tiempo, lujo), y `TaxiDispatcher` encarga el taxi elegido (Martin, 2017, p. 242).

Tras más de un año en producción, Marketing anuncia un servicio de entrega de gatitos. La empresa tendrá puntos de recogida, y un taxi cercano recogerá el gatito y lo llevará a la dirección indicada. Solo participa, por ahora, uno de los proveedores. Además, ningún conductor alérgico a los gatos debe ser elegido para estas entregas, y a los clientes que declaren esa alergia no se les debe asignar un vehículo que haya transportado gatitos en los últimos tres días. La pregunta de Martin es cuántos servicios hay que cambiar, y la respuesta es todos. El desarrollo y el despliegue tendrán que coordinarse con cuidado; es decir, los servicios están acoplados y no se pueden desarrollar, desplegar ni mantener de forma independiente (Martin, 2017, p. 243). La figura 11 muestra dónde cae el cambio en cada servicio. Ese reparto por servicio es una elaboración nuestra a partir de la descripción de Martin.

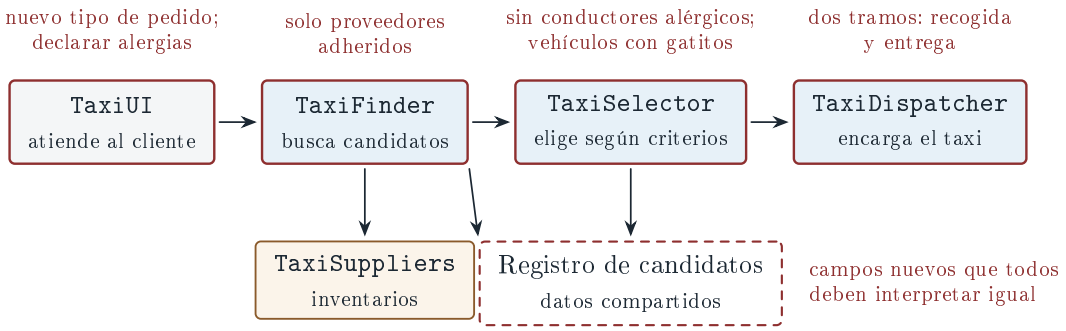


Figura 11: Descomposición funcional del agregador de taxis y alcance del requisito de los gatitos: deben cambiar todos los servicios del diagrama y también el registro que comparten. Los proveedores se representan como sistemas externos. Elaboración propia a partir de Martin (2017, pp. 242–243); el reparto del cambio por servicio es nuestro.

5.2.2. ¿Por qué la división en servicios por verbos o entidades llevó a este problema?

Porque es una descomposición funcional y el nuevo requisito es transversal. `TaxiFinder`, `TaxiSelector` y `TaxiDispatcher` son los pasos de un único flujo (verbos), y en cada paso la lógica genérica está mezclada con la que es propia de los viajes, que es justo la parte que Martin extrae después a un componente `Rides` (Martin, 2017, p. 244). La entrega de gatitos no añade un paso nuevo: cambia qué significa cada paso. Martin lo resume así: es el problema de las preocupaciones transversales, y las descomposiciones funcionales como esta son muy vulnerables a funcionalidades nuevas que atraviesan todos sus comportamientos (Martin, 2017, p. 243). En términos de principios, el diseño separó lo que cambia por la misma razón. El principio de cierre común pide reunir en un componente las clases que cambian por las mismas razones y al mismo tiempo, y separar las que cambian en momentos distintos y por razones distintas (Martin, 2017, p. 105). Aquí las reglas de un mismo producto quedaron repartidas entre cuatro servicios.

Porque los servicios siguen acoplados por los datos que comparten. Que cada servicio corra en su propio proceso los desacopla a nivel de variables, pero no de datos. Si se añade un campo a un registro que viaja entre servicios, todos los que lo usan deben cambiar y ponerse de acuerdo en cómo interpretarlo (la falacia del desacoplamiento).

Mientras haya acoplamiento por datos o por comportamiento, el desarrollo y el despliegue tienen que coordinarse (la falacia del desarrollo y despliegue independientes) (Martin, 2017, pp. 240–241). El registro de candidatos y el formato del pedido son ese punto de acoplamiento.

Porque servicio no es sinónimo de frontera arquitectónica. Para Martin, la arquitectura la definen las fronteras que separan la política de alto nivel de los detalles y que respetan la Regla de Dependencia; los servicios que solo separan comportamientos de la aplicación son poco más que llamadas a funciones caras (Martin, 2017, p. 240). Partir por entidades (un `TaxiManager`, un `DriverManager`) tampoco lo resuelve. Richards y Ford lo llaman la trampa de las entidades: crear un componente por entidad no es una arquitectura, sino un mapeo de componentes a la base de datos, y suele revelar que no se ha pensado en los flujos reales del negocio (Richards & Ford, 2020, cap. 8). Por eso advierten que los microservicios no deben modelarse simplemente a semejanza de entidades individuales de la base de datos (Richards & Ford, 2020, cap. 17). Tanto si se parte por pasos como por entidades, las fronteras siguen la estructura técnica o de datos y no los ejes por los que cambia el negocio.

5.2.3. ¿Cómo diseñarlo con DDD y Onion/Clean para que el cambio afecte a un solo componente?

DDD: fronteras por contexto, no por paso. Los microservicios se inspiran en el contexto delimitado de DDD. Dentro de un contexto, el código y los esquemas de datos están acoplados entre sí, pero nunca a nada de otro contexto, ni a su base de datos ni a sus clases; así cada contexto define solo lo que necesita (Richards & Ford, 2020, cap. 17). Aplicado al agregador, «viaje» y «entrega de gatito» son dos contextos con lenguaje y reglas propios: puntos de recogida, adhesión de proveedores y alergias declaradas solo tienen sentido en el segundo. Lo que comparten es una política genérica (encontrar vehículos, elegir uno y encargarlo), que pasa a ser el núcleo estable. Las fronteras se trazan donde hay un eje de cambio: los componentes de un lado cambian a otro ritmo y por otras razones que los del otro lado (Martin, 2017, p. 173). Aquí el eje de cambio es el producto.

Onion/Clean: el núcleo define las abstracciones y los productos las implementan. En Onion, el código puede depender de capas más centrales, pero nunca de capas más externas; el núcleo necesita implementaciones de sus interfaces, que residen en los bordes y se inyectan en tiempo de ejecución (Palermo, 2008). La Regla de Dependencia de Clean dice lo mismo: las dependencias del código fuente solo apuntan hacia las políticas de más alto nivel (Martin, 2017, p. 203). Martin aplica la misma técnica a cualquier frontera: unos componentes contienen las reglas de negocio centrales, otros son plugins con funciones que no forman parte del núcleo, y todas las flechas apuntan hacia el núcleo (Martin, 2017, p. 173). Martin lo resuelve primero con objetos: la mayor parte de la lógica de los servicios originales se conserva en las clases base del modelo, la lógica propia de los viajes se extrae a un componente `Rides` y la entrega de gatitos va en un componente `Kittens`. Ambos sobrescriben las clases base abstractas mediante *Template Method* o *Strategy* (Martin, 2017, p. 244). Con este esquema, al implementar los gatitos cambia `TaxiUI`, pero nada más: basta añadir un `.jar` (o una *gem*, o una DLL) que se carga en tiempo de ejecución, y la funcionalidad queda desacoplada y se puede desarrollar y desplegar de forma independiente (Martin, 2017, p. 244). Después aplica la misma estructura

dentro de cada servicio: un servicio puede verse como un conjunto de clases abstractas en uno o más `.jar`, y cada funcionalidad nueva, como otro `.jar` que las extiende (Martin, 2017, pp. 245–246).

Qué significa aquí «un solo componente o servicio». Con el diseño de Martin, el cambio es un único componente lógico, `Kittens`, que se despliega como un `.jar` en cada servicio, y el único servicio que se modifica es `TaxiUI`. La otra opción, coherente con DDD, es un servicio propio de entrega de gatitos: para Richards y Ford, las fronteras de un microservicio deben capturar un dominio o un flujo de trabajo (Richards & Ford, 2020, cap. 17). Ese servicio reutilizaría la política genérica de asignación a través de contratos publicados, o duplicaría la parte que necesite. Esa duplicación es aceptable cuando el objetivo es un alto desacoplamiento, porque se prefiere la duplicación a la reutilización (Richards & Ford, 2020, cap. 17). En ambos casos, el requisito nuevo queda dentro de una sola frontera.

5.2.4. Propuesta: `Rides` y `Kittens` como plugins

La figura 12 aplica esta idea a los tres servicios. Cada servicio conserva su núcleo (clases abstractas y puntos de extensión) y recibe la variante de cada producto como componente separado: en `TaxiFinder`, qué proveedores participan; en `TaxiSelector`, qué vehículos son admisibles y cómo se ordenan; en `TaxiDispatcher`, cuántos tramos tiene el encargo. Como indica Martin, las fronteras arquitectónicas no caen entre los servicios, sino que los atraviesan y los dividen en componentes (Martin, 2017, p. 246). Las clases concretas las crean fábricas bajo el control de la interfaz de usuario (Martin, 2017, p. 244). En nuestra lectura, ese papel corresponde al componente `Main`: según Martin, su tarea es crear todas las fábricas, estrategias y demás recursos globales y luego entregar el control a las partes abstractas de alto nivel del sistema (Martin, 2017, p. 232), y se puede pensar como un plugin de la aplicación (Martin, 2017, p. 237).

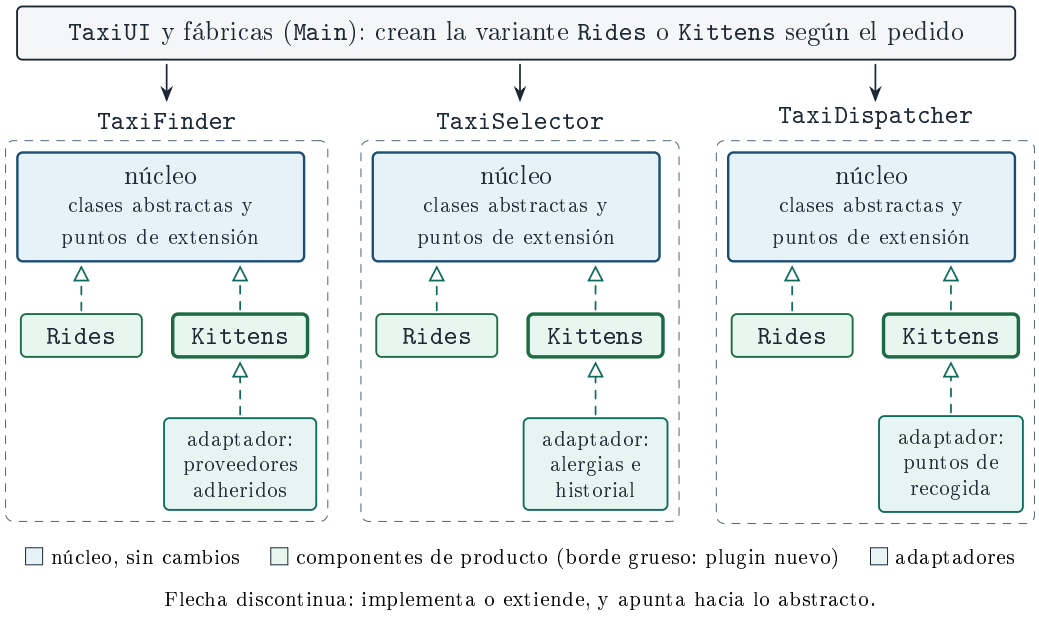


Figura 12: Diseño basado en componentes. Cada servicio separa un núcleo estable de los componentes de producto; `Kittens` se añade como plugin en cada servicio y sus puertos los implementan adaptadores. Elaboración propia a partir de Martin (2017, pp. 244–247) y Cockburn (2005).

La parte más delicada es la regla de las alergias, porque no afecta solo a las entregas: también cambia la elección de vehículo en los viajes normales de los clientes alérgicos. Para que eso no obligue a tocar `Rides`, el núcleo de `TaxiSelector` ofrece un segundo punto de extensión: una lista de reglas de admisibilidad que aportan los plugins instalados. El fragmento de la figura 13 lo ilustra. El núcleo no nombra ni `Rides` ni `Kittens`; `Kittens` aporta su regla y guarda sus propios datos (alergias declaradas, historial de entregas) detrás de puertos, en lugar de añadir campos a los registros que comparten todos los servicios.

```
// ---- Núcleo de TaxiSelector: estable; no conoce Rides ni Kittens ----
public interface EligibilityRule {                  // punto de extensión
    boolean allows(Vehicle vehicle, Order order);
}
public abstract class TaxiSelector {                // Template Method
    private final List<EligibilityRule> rules;      // aportadas por los plugins
    protected TaxiSelector(List<EligibilityRule> rules) { this.rules = rules; }
    public final Optional<Vehicle> select(List<Vehicle> candidates, Order order) {
        return candidates.stream()
            .filter(v -> rules.stream().allMatch(r -> r.allows(v, order)))
            .min(ranking(order));         // paso variable: lo da cada producto
    }
    protected abstract Comparator<Vehicle> ranking(Order order);
}

// ---- Componente Kittens (un .jar nuevo): extiende sin tocar el núcleo ----
public final class KittenAllergyRule implements EligibilityRule {
    private final AllergyRegistry allergies;        // puerto propio de Kittens
    private final KittenDeliveryLog deliveries;     // puerto propio de Kittens
    // (constructor que recibe ambos puertos omitido)
    @Override
    public boolean allows(Vehicle vehicle, Order order) {
        if (order instanceof KittenOrder)  // entrega: conductor no alérgico
            return !allergies.isAllergic(vehicle.driver());
        return !(allergies.isAllergic(order.customer())  // viaje de cliente
                 && deliveries.carriedKittenWithin(vehicle, Duration.ofDays(3)));
    }
}
```

Figura 13: Fragmento ilustrativo en Java (el ejemplo de Martin habla de `.jar` y clases abstractas). `RideSelector` y `KittenSelector` extienden `TaxiSelector` con su propio orden de preferencia; las fábricas de `Main` registran las reglas de todos los plugins instalados. Elaboración propia.

5.2.5. Cómo respeta la solución el OCP y qué papel juegan los adaptadores

Principio de abierto/cerrado. Un artefacto de software debe estar abierto a la extensión y cerrado a la modificación: su comportamiento debe poder ampliarse sin modificarlo (Martin, 2017, p. 70). En la solución, añadir los gatitos consiste en desplegar componentes nuevos con clases que extienden las abstracciones del núcleo, no en redesplegar servicios modificados. Basta añadir los nuevos `.jar` a las rutas de carga de los servicios, y por eso la ampliación cumple el OCP (Martin, 2017, p. 246). A escala de arquitectura, el OCP se consigue repartiendo el sistema en componentes y ordenándolos en una jerarquía de dependencias que proteja a los de más alto nivel de los cambios en los de más bajo nivel (Martin, 2017, p. 75). El núcleo y `Rides` quedan cerrados; el sistema, abierto.

Hay que matizar que el cierre nunca es total. El propio Martin reconoce que `TaxiUI` debe cambiar (Martin, 2017, p. 244), y en otra obra advierte que, por cerrado que esté un módulo, siempre habrá algún cambio frente al que no lo esté. Como el cierre no puede ser completo, tiene que ser estratégico: el diseñador elige frente a qué cambios protegerse y construye las abstracciones para ello (Martin, 2014, p. 105). En este caso, si el núcleo de `TaxiDispatcher` solo supiera encargar viajes de un tramo, el primer producto de dos tramos obligaría a modificarlo una vez para abrir ese punto de extensión. A partir de ahí, productos parecidos, como la entrega de paquetes, serían extensiones puras.

Papel de los adaptadores. En la arquitectura hexagonal, un puerto identifica una conversación con un propósito, y suele haber varios adaptadores por puerto, uno por cada tecnología que se conecta a él (Cockburn, 2005, sec. «Nature of the Solution»). En la extensión de los gatitos, los adaptadores cumplen tres funciones:

1. **Conectan el plugin con el exterior sin contaminar el núcleo.** `Kittens` necesita conversar con actores nuevos: el registro de puntos de recogida, los datos de adhesión de los proveedores, las alergias declaradas y el historial de entregas. Define un puerto para cada uno (`CollectionPoints`, `ParticipatingSuppliers`, `AllergyRegistry` y `KittenDeliveryLog`), y los implementan adaptadores de infraestructura, por ejemplo uno SQL o uno que consulte la API de un proveedor (figura 12). El núcleo y `Rides` no se enteran.

2. **Hacen posible probar el plugin aislado.** El mismo puerto admite un adaptador en memoria para las pruebas (Cockburn, 2005, sec. «Nature of the Solution»), así que las reglas de `KittenAllergyRule` se verifican sin base de datos ni proveedores reales.

3. **Concentran el único cambio inevitable.** `TaxiUI` es un adaptador primario, es decir, del lado de los actores que conducen la aplicación (Cockburn, 2005, sec. «The Left-Right Asymmetry»). Añade el nuevo tipo de pedido y la declaración de alergias y, junto con las fábricas, decide qué implementación usar. El cambio queda en el borde, que es donde debe estar.

5.2.6. Síntesis

- El *Kitty Problem* (Martin, 2017, pp. 242–244) muestra que un requisito transversal obliga a cambiar y desplegar a la vez todos los servicios de una descomposición funcional.

- El origen es partir por pasos (verbos) o por entidades. Así se reparten entre servicios las reglas que cambian juntas, y los servicios siguen acoplados por los datos que comparten.

- Con DDD y Onion/Clean, las fronteras siguen los ejes de cambio del negocio: un núcleo estable con puntos de extensión y un componente por producto (`Rides`, `Kittens`), con todas las dependencias apuntando hacia el núcleo.

- Los gatitos se añaden como plugins sin modificar el núcleo ni `Rides`, lo que cumple el OCP. El cierre es estratégico y no total: `TaxiUI` cambia, y los adaptadores conectan el plugin con sus fuentes de datos y con las pruebas sin tocar el núcleo.

6. N-Capas y atributos de calidad

6.1. Enunciado

La Arquitectura N-Capas es criticada por su bajo rendimiento en características como «Agilidad general», «Facilidad de despliegue» y «Escalabilidad» (Richards, 2015). Un estudiante propone que, con el uso de capas abiertas (*open layers*) y técnicas como el «fast-lane reader», se pueden mitigar estos problemas.

Analice esta afirmación. ¿Cómo afecta la introducción de capas abiertas al principio de Aislamiento de Capas y a la mantenibilidad a largo plazo? ¿Por qué la solución de capas abiertas puede llevar a un «sinkhole anti-pattern» y qué dice la regla del 80/20 sobre esto? Finalmente, explique por qué, para un sistema con requisitos de alta escalabilidad y agilidad, una arquitectura como la Hexagonal o la Limpia, que se basan en los principios de DDD y desacoplamiento, son inherentemente más adecuadas que una Arquitectura N-Capas, incluso con optimizaciones.

6.2. Respuesta

6.2.1. Capas abiertas y *fast-lane reader*: beneficio local, coste estructural

La Arquitectura N-Capas organiza el sistema en capas horizontales (presentación, negocio, persistencia) con dependencias que fluyen de arriba hacia abajo. Su principal fortaleza es la simplicidad; su principal debilidad, que cada petición debe atravesar todas las capas, incluidas aquellas que solo transportan datos sin ejecutar lógica relevante (Richards, 2015). Las **capas abiertas** relajan esa regla: permiten que una petición salte capas intermedias. El caso típico, que el enunciado denomina *fast-lane reader*, es una lectura simple en la que la presentación accede directamente a la de persistencia, evitando el recorrido por la capa de negocio (Richards & Ford, 2020).

El beneficio es real pero acotado: mejora el rendimiento en lecturas simples y reduce la latencia artificial de las capas que solo hacen *pass-through*. El coste también es real: se pierde el **aislamiento de capas**, que es la segunda ventaja que la arquitectura ofrecía, después de la simplicidad. Si la presentación conoce la persistencia, un cambio de esquema o de motor impacta directamente en la UI, y la evolución independiente de cada capa deja de ser posible: la mantenibilidad a largo plazo se deteriora (Richards & Ford, 2020). La figura 14 contrasta ambos escenarios.

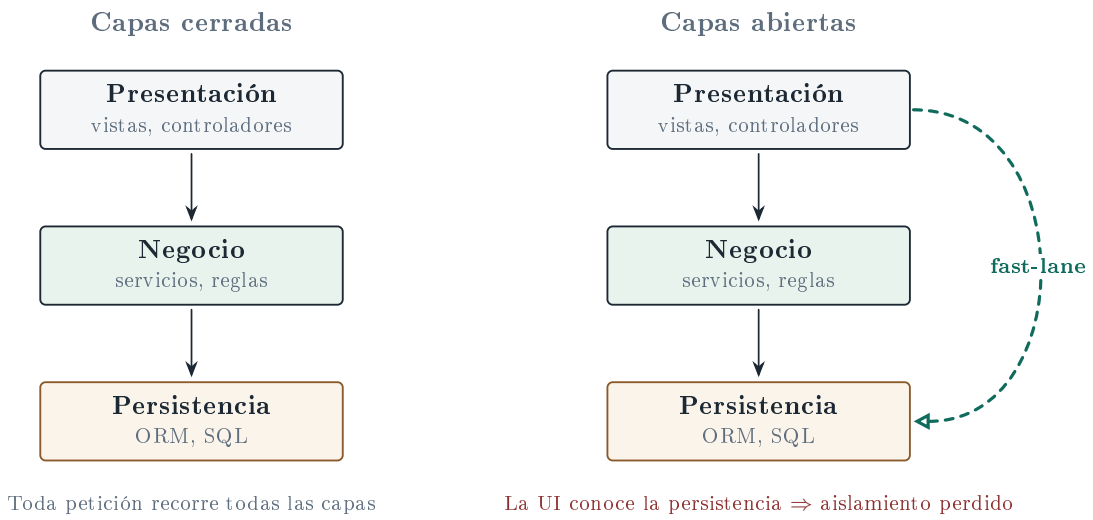


Figura 14: Capas cerradas frente a capas abiertas. A la derecha, el atajo *fast-lane* conecta presentación con persistencia sin pasar por negocio: se gana velocidad en lecturas simples a cambio del aislamiento de capas. Elaboración propia a partir de (Richards, 2015; Richards & Ford, 2020).

6.2.2. El *sinkhole anti-pattern* y la regla 80/20

El *sinkhole anti-pattern* aparece cuando las peticiones atraviesan varias capas como simple *pass-through*: la presentación recibe la petición, la pasa al negocio, el negocio la pasa a la persistencia, y ninguna capa ejecuta lógica de negocio relevante (Richards, 2015). Abrir capas indiscriminadamente no resuelve este problema: acelera el tráfico que lo constituye y lo enmascara tras una mejora de rendimiento, de modo que el diseño queda igual de pobre pero más difícil de gobernar, porque ya no hay una frontera clara que diga dónde vive cada responsabilidad.

Hay además una relación causal entre capas abiertas y *sinkhole*. Al permitir que la presentación acceda directamente a la persistencia, se elimina la frontera que forzaba a la capa de negocio a justificar su existencia; sin esa frontera, nada impide que el negocio se degrade a mero *pass-through*. Las capas abiertas no solo enmascaran el patrón, sino que incentivan el diseño que lo produce.

La **regla 80/20** que propone Richards (2015) funciona como criterio de diagnóstico. Si un 20 % o menos de las peticiones son *sinkholes*, la arquitectura es aceptable y la optimización es prescindible: no hay razón para romper el aislamiento de capas por una minoría de casos. Si un 80 % o más lo son, la propia regla dictamina que N-Capas no es el estilo adecuado para ese dominio; acelerar el tráfico no corrige el diagnóstico. En ninguno de los dos casos la propuesta del enunciado resuelve la ausencia de lógica de negocio: solo acelera el tráfico que recorre un tramo que sigue mal diseñado. La figura 15 ilustra ambos casos.

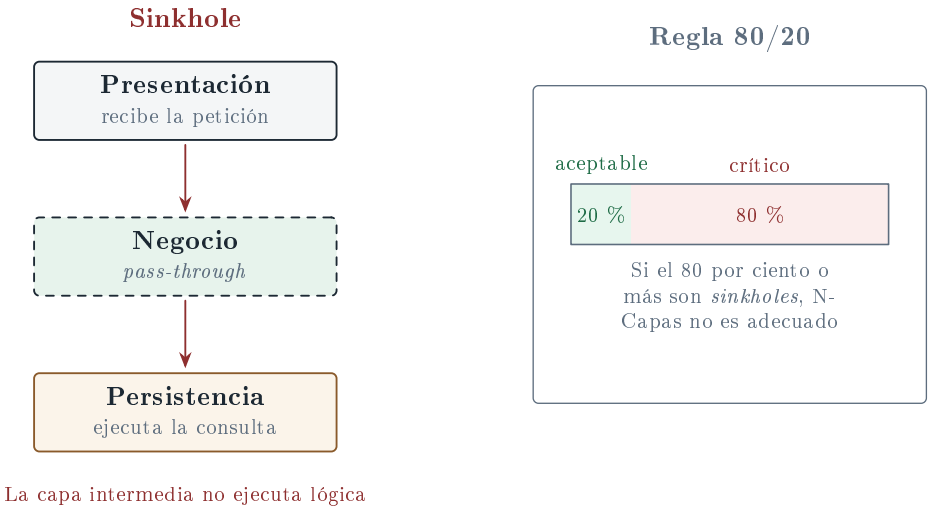


Figura 15: *Sinkhole anti-pattern* y regla 80/20. Cuando la mayoría de las peticiones solo atraviesan capas sin lógica, el problema no es de rendimiento sino de diseño. Elaboración propia a partir de (Richards, 2015).

6.2.3. Por qué Hexagonal y Clean son inherentemente más adecuadas

En N-Capas la partición es **técnica**, no por dominio: un mismo dominio de negocio (inventario, pedidos, facturación) queda repartido por todas las capas, de modo que cualquier cambio en un dominio obliga a tocar presentación, negocio y persistencia a la vez. La figura 16 contrasta esa partición horizontal con la partición por dominio que proponen DDD y las arquitecturas derivadas. Esa diferencia de partición explica por qué la evaluación de atributos de calidad de N-Capas es baja en **escalabilidad**, **desplegabilidad** y **testabilidad** (Richards, 2015), y por qué la partición por dominio es la que adoptan las arquitecturas orientadas al dominio (De la Torre Llorente et al., 2010).

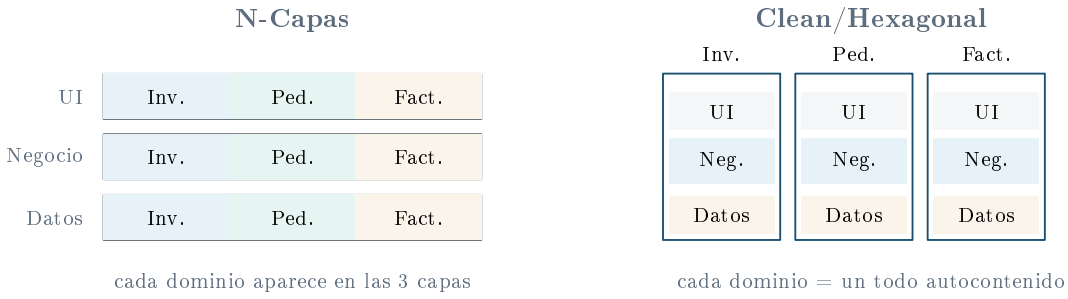


Figura 16: Partición técnica frente a partición por dominio. En N-Capas cada dominio queda repartido por todas las capas (bandas); en Clean/Hexagonal cada dominio es autocontenido (columnas) y evoluciona sin arrastrar a los demás. Elaboración propia.

DDD ataca ese problema desde el modelado: los **Bounded Contexts** delimitan dominios con su propio Lenguaje Ubicuo y su propio modelo, de modo que cada contexto evoluciona sin arrastrar a los demás. La guía de arquitectura N-Capas orientada al dominio aplica principios de DDD para separar el dominio de la infraestructura (De la Torre Llorente et al., 2010). Clean, Hexagonal y Onion proporcionan la estructura técnica para materializar esos contextos: cada Bounded Context se implementa como un núcleo aislado, con su dominio en el centro y sus adaptadores en el borde, y las dependencias apuntando **solo hacia adentro** (Cockburn, 2005; Martin, 2017; Palermo, 2008). DDD

dice *qué* aislar; Clean, Hexagonal y Onion dicen *cómo* hacerlo sin que la infraestructura contamine el núcleo.

Esto no es una optimización cosmética: **elimina estructuralmente** el acoplamiento que N-Capas intenta mitigar con capas abiertas. La figura 17 contrasta ambos flujos: en N-Capas la cadena de dependencias es UI → Dominio → Persistencia; en Clean/Hexagonal la UI y la persistencia dependen ambas del dominio, que no depende de ninguna. El dominio se prueba y evoluciona sin conocer la base de datos, la UI ni el framework, y los detalles tecnológicos son sustituibles sin reescribir reglas de negocio (Cockburn, 2005; Martin, 2017; Palermo, 2008). Esa guía insiste en la misma separación: el dominio debe ser independiente de la tecnología de persistencia y de los mecanismos de entrega (De la Torre Llorente et al., 2010).

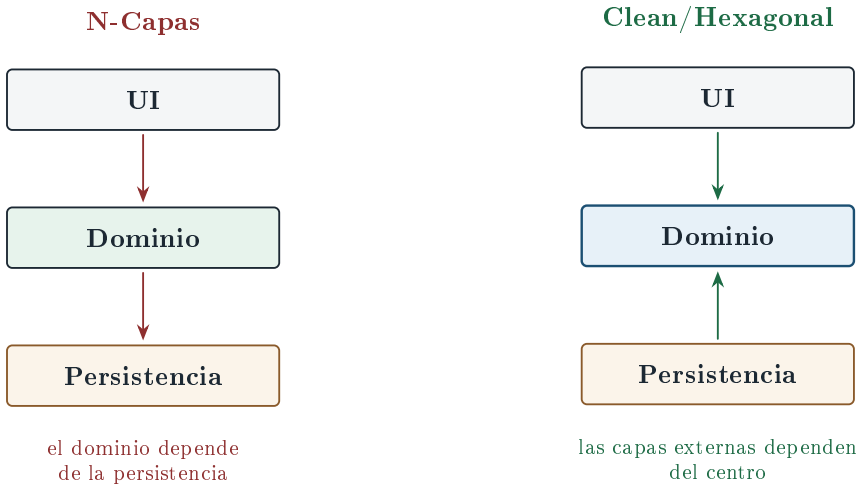


Figura 17: Dirección de las dependencias. A la izquierda, el dominio conoce la persistencia (cadena hacia abajo). A la derecha, la UI y la persistencia convergen sobre el dominio (las capas externas dependen del centro). La inversión no añade capas: cambia quién conoce a quién. Elaboración propia a partir de (Cockburn, 2005; Martin, 2017; Palermo, 2008).

Por eso, para requisitos de **alta escalabilidad y agilidad**, estas arquitecturas son inherentemente más adecuadas: no dependen de abrir o cerrar capas para sortear un acoplamiento que su propio diseño ya evita. La separación en Bounded Contexts permite que cada contexto evolucione y se pruebe de forma independiente; si además se despliega como servicio separado, también escala de forma independiente. La agilidad mejora porque un cambio en un contexto no obliga a tocar los demás; la testabilidad, porque el núcleo se prueba sin infraestructura (Richards, 2015; Richards & Ford, 2020).

6.2.4. Síntesis

La propuesta del enunciado es parcialmente válida: las capas abiertas y el *fast-lane reader* mejoran el rendimiento en lecturas simples, pero lo hacen a costa del aislamiento de capas, que es una de las ventajas de N-Capas. Además, abrir capas **enmascara** el *sinkhole anti-pattern* en lugar de corregirlo, e incluso lo incentiva al eliminar la frontera que obligaba a la capa de negocio a justificar su existencia. La regla 80/20 muestra que, cuando la mayoría de las peticiones son *pass-through*, el problema no es de rendimiento sino de diseño: N-Capas no es el estilo adecuado. Hexagonal y Clean no optimizan N-Capas: la sustituyen por una partición por dominio —los Bounded Contexts de DDD—

con dependencias hacia adentro, que elimina estructuralmente el acoplamiento en lugar de sortearlo con atajos. Para alta escalabilidad y agilidad, esa diferencia de diseño importa más que cualquier optimización local de capas.

Referencias bibliográficas

Cockburn, A. (2005). *Hexagonal Architecture (Ports and Adapters)* [Consultado para el seminario]. Consultado el 1 de enero de 2026, desde https://alistair.cockburn.us/hexagonal-architecture/

De la Torre Llorente, C., Zorrilla Castro, U., Ramos Barroso, M. A., & Calvarro Nelson, J. (2010). *Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0 (Borrador)* (inf. téc.). Microsoft Ibérica.

Martin, R. C. (2014). *Agile Software Development, Principles, Patterns, and Practices* (Pearson New International ed.). Pearson Education.

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Palermo, J. (2008). *The Onion Architecture*. Consultado el 1 de enero de 2026, desde https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/

Richards, M. (2015). *Software Architecture Patterns*. O'Reilly Media.

Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.

Declaración de Uso de la Inteligencia Artificial

Durante la preparación de este documento se recurrió a distintas herramientas de inteligencia artificial: Claude (Anthropic), mediante el modelo Claude Opus 5.5; ChatGPT (OpenAI), mediante el modelo GPT 6 Luna; y Gemini (Google), en su versión más reciente, empleada a través de Gemini Notebook. Estas herramientas se utilizaron principalmente para resumir y facilitar la comprensión de la información contenida en la bibliografía recomendada, cuyas fuentes presentan una extensión considerable.

Los integrantes del equipo obtuvieron el contenido de este trabajo a partir del estudio de las fuentes bibliográficas recomendadas y se encargaron de redactarlo. Después, utilizaron la inteligencia artificial como apoyo para enriquecer los textos y perfeccionar su redacción.

Respecto a los diagramas, Olivia Ortiz Arboláez empleó Gemini Notebook para generar las imágenes correspondientes a los diagramas que tenía asignados, mientras que Anthony Cruz Garcia y Enrique González González utilizaron ChatGPT, en el modelo GPT 6 Luna, para elaborar los que les correspondían.

Por su parte, Jose Manuel Porras Guerra utilizó Claude (Anthropic), modelo Claude Opus 5.5, únicamente como herramienta de apoyo en la elaboración de la respuesta a la pregunta 5: le sirvió para localizar en la bibliografía orientada los pasajes pertinentes y sus páginas, y para contrastar posibles estructuras de la respuesta y un borrador inicial del texto, de las figuras y del código correspondiente. Dicho borrador fue únicamente un punto de partida: el integrante lo revisó, lo contrastó con las fuentes citadas y lo adaptó, de modo que el análisis y las decisiones finales son suyas.

Todo el contenido fue revisado, analizado y verificado conjuntamente por los integrantes del equipo, quienes asumen la responsabilidad final sobre el documento. Aunque se empleó la inteligencia artificial como herramienta de apoyo, los integrantes conocen y dominan los temas abordados en cada una de las preguntas.